

```

import matplotlib.pyplot as plt
import numpy as np

# Define the function
def f(x):
    return (x + 3)**2

# Define its derivative (gradient)
def grad_f(x):
    return 2 * (x + 3)

# Gradient Descent Implementation
def gradient_descent(start_x=2, learning_rate=0.1, max_iter=50,
tol=1e-6):
    x = start_x
    x_history = [x]

    for i in range(max_iter):
        gradient = grad_f(x)
        x_new = x - learning_rate * gradient

        x_history.append(x_new)

        if abs(x_new - x) < tol: # convergence check
            break

    x = x_new

    return x, f(x), x_history

# Run Gradient Descent
min_x, min_y, x_steps = gradient_descent()
print("Local minima at x =", min_x)
print("Minimum value y =", min_y)

Local minima at x = -2.9999286376153647
Minimum value y = 5.092589940842615e-09

# Visualization
x_vals = np.linspace(-6, 2, 100)
y_vals = f(x_vals)

plt.plot(x_vals, y_vals, label='y = (x+3)^2')
plt.scatter(x_steps, [f(x) for x in x_steps], color='red',
label='Gradient Descent Steps')
plt.title("Gradient Descent to find Local Minima")
plt.xlabel("x")
plt.ylabel("y")
plt.legend()
plt.show()

```

Gradient Descent to find Local Minima

